

# Learning Accurate Robot Dynamics From Position-Only Data With Discrete Lagrangian Neural Networks

Zhiming Li<sup>1</sup>, Fengyu Sun<sup>2</sup>, Shuangshuang Wu<sup>3</sup>, Fuchun Sun<sup>4</sup>, *Fellow, IEEE*, Peilin Xiong, Cong Liu, and Wenbai Chen<sup>5</sup>

**Abstract**—Achieving high-fidelity modeling of robot dynamics remains a major challenge in scientific computing and artificial intelligence. Black-box models are prone to overfitting and lack interpretability, while continuous Lagrangian-based Physics-Informed Neural Networks (PINNs) require differentiated data from position measurements, increasing acquisition complexity and introducing noise that degrades performance. To address these issues, we propose a Structured Discrete Lagrangian Neural Network (StrDLNN), grounded in the discrete Lagrangian formulation and requiring only position data. Experiments demonstrate that StrDLNN recovers the underlying physical structure from position-only observations, achieving superior data efficiency, extrapolation capability, and robustness to sampling frequency compared with existing baselines. When applied to a UR10e robot, StrDLNN incorporates a friction model and accurately identifies both overall dynamics and friction parameters without friction labels, achieving a coefficient of determination  $R^2$  of 0.94 in dynamics modeling.

**Index Terms**—Dynamics, deep learning methods, friction modeling, physics-informed neural networks (PINNs).

## I. INTRODUCTION

ACCURATE modeling of robot dynamics and friction is essential for high-performance robot control and operation [1]. Therefore, achieving high-fidelity representations of complex physical dynamics remains a major challenge in scientific computing and artificial intelligence [2]. Traditional Newton–Euler and Euler–Lagrange formulations require physical parameters such as mass, inertia, and center of mass, which

are often difficult to obtain and are frequently withheld by manufacturers for intellectual property reasons [3].

Data-driven parameter identification methods, like least squares, fit parameters to observed data in the equations of motion [4]. However, noise and modeling assumptions can lead to physically inconsistent results, such as negative masses or non-positive-definite inertia matrices. To address this, Gautier et al. reformulated the Recursive Newton–Euler Algorithm (RNEA) into a differentiable computation graph with physical constraints, proposing DiffNEA for physically consistent parameters learning [5]. Despite this, the nonlinear nature of robot dynamics still limits the accuracy of existing methods.

Data-driven black-box methods can model robot dynamics from state-control trajectory data, with neural networks replacing traditional models due to their universality, scalability, and representational power. While black-box models such as multi-layer perceptrons (MLP) provide high-fidelity approximations, they are prone to overfitting, lack interpretability, and struggle to incorporate physical knowledge. In robotics, ignoring physical priors can lead to spurious correlations, unpredictable behavior, and compromised safety and performance [2].

In recent years, Physics-Informed Neural Networks (PINNs) have combined deep learning with physical principles, enabling data-driven modeling under physical constraints for robotic dynamics [6]. Lagrangian mechanics describe dynamical systems and can be integrated into the structure of a neural network. In this context, Deep Lagrangian Networks (DeLaN) [7] and Lagrangian Neural Networks (LNN) [8] integrate continuous Lagrangian dynamics with deep learning to learn robot dynamics end-to-end, improving physical consistency and generalization.

Continuous Lagrangian dynamics require first and second-order derivatives of position  $q$ , making methods like DeLaN and LNN dependent on accurate velocity  $\dot{q}$  and acceleration  $\ddot{q}$  data [2]. This is particularly challenging in system identification tasks relying solely on positional data, such as motion capture, pixel data, or low-resolution tracking [9]. In many physical systems (e.g., low-cost robotic arms [10], excavators [11]), only position data is available, requiring velocity and acceleration estimation via finite differences, which complicates data acquisition and introduces noise and truncation errors, limiting accuracy [12].

These challenges motivate the use of discrete Lagrangian dynamics, which relies solely on position input and provides a simpler, more efficient computation. Discrete Lagrangian systems can accurately model continuous motions, provided the step-size is small enough, thereby justifying their use from a modeling perspective [13]. Hansen et al. proposed the Discrete

Received 5 September 2025; accepted 24 December 2025. Date of publication 19 January 2026; date of current version 27 January 2026. This article was recommended for publication by Associate Editor C. Sun and Editor S.-J. Chung upon evaluation of the reviewers' comments. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0114505, in part by the National Natural Science Foundation of China under Grant 62276028, in part by the Major Research Plan of the National Natural Science Foundation of China under Grant 92267110, and in part by Beijing Municipal Natural Science Foundation–Xiaomi Innovation Joint Fund under Grant L233006. (Corresponding author: Fuchun Sun.)

Zhiming Li is with the School of Artificial Intelligence, Shandong University, Jinan 250100, China (e-mail: lzming@mail.sdu.edu.cn).

Fengyu Sun, Shuangshuang Wu, Peilin Xiong, and Wenbai Chen are with the College of Automation, Beijing Information Science and Technology University, Beijing 100192, China (e-mail: chenwb@bistu.edu.cn).

Fuchun Sun is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: fc-sun@mail.tsinghua.edu.cn).

Cong Liu is with the School of Mechanical and Electrical Engineering, China University of Mining and Technology, Beijing 100083, China.

Digital Object Identifier 10.1109/LRA.2026.3655271

Forced Lagrangian Neural Network (DFLNN), which applies discrete forced Lagrangian dynamics to predict a system's Lagrangian [9]. However, it relies on a single network to regress the Lagrangian without accounting for the robot's structure, limiting its applicability in complex systems. We therefore propose the Structured Discrete Lagrangian Neural Network (StrDLNN), which, instead of directly modeling the Lagrangian, first models the system's potential energy and mass matrix, and then derives the Lagrangian. This approach directly extracts the underlying dynamics from position data while incorporating a friction model to enable modeling of complex robotic systems. The main contributions are:

- We propose the StrDLNN model, incorporating discrete Lagrangian dynamics into a physics-informed framework. By modeling the mass matrix and potential energy, it better captures robot dynamics and enhances physical consistency and interpretability.
- We experimentally demonstrate the ability of StrDLNN to learn the underlying physical structure using only position observations, showing superior data efficiency, extrapolation capability, and robustness to sampling frequency compared to existing baselines.
- We validate StrDLNN on the UR10e Robot, exploring its optimal integration with energy dissipation mechanisms, and show that it can incorporate friction modeling and accurately learn overall dynamics and friction parameters, without the need for friction torque labels.

## II. RELATED WORK

Data-driven system identification uses analytical equations of motion as the transfer function, with model parameters learned from observed data. This approach requires knowledge of the system's kinematic chain and high-quality data for effective identification [6]. In contrast, Lagrangian-based data-driven methods [1], [2], [6], [7], [8], [10], [11], [14], [15], [16], [17], [18] model physical systems using the Euler-Lagrange equations in generalized coordinates  $q$  and velocities  $\dot{q}$ , with the Lagrangian  $\mathcal{L}(q, \dot{q})$  defined as the difference between kinetic and potential energy. These methods do not require specific system knowledge but still capture comparable characteristics to system identification.

As shown in Fig. 1(a), there are generally two approaches to modeling  $\mathcal{L}(q, \dot{q})$ . One method models a structured Lagrangian  $\mathcal{L}$  by separately representing kinetic and potential energy, as in DeLaN [7], while the other models an unstructured Lagrangian  $\mathcal{L}$  with a single network, as in LNN [8]. Both are effective for conservative systems, but lacking energy dissipation mechanisms, Lagrangian mechanics struggles to capture non-conservative forces such as joint friction, limiting its applicability to general physical systems.

To address dissipative effects, existing extensions generally follow three approaches: using black-box models to capture input-dependent non-conservative forces [2], [11], learning an additional damping matrix [15], [19], or incorporating a parametric friction model [1], [10], [14]. These components are often trained jointly with Lagrangian-based data-driven methods to enhance the ability to represent non-conservative forces, and this approach has been widely applied in robotic dynamics modeling tasks, including industrial robots [1], soft robots [15], compliant manipulators [2], excavators [11], ground vehicles [16] and unmanned uncrewed aerial vehicles [17], [19]. Since joint friction

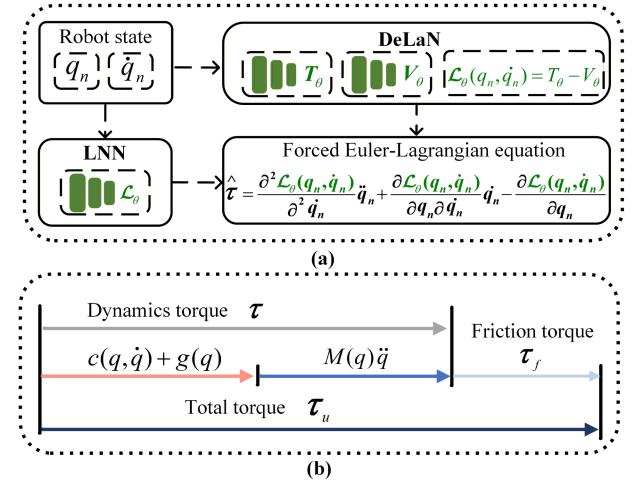


Fig. 1. (a) Architecture of DeLaN and LNN. Trainable components are highlighted in green. (b) The structure of inverse dynamics is illustrated by a one-dimensional vector diagram.

is the main source of non-conservative forces in robotic manipulators, dynamic modeling often incorporates learned friction models to better capture non-conservative forces [14].

Existing continuous Lagrangian-based data-driven methods typically require velocity and acceleration inputs for inverse dynamics, but most robotic systems can only acquire position data [12], necessitating numerical differentiation that increases complexity and introduces noise, reducing accuracy and stability. The discrete Lagrangian formulation eliminates this need [9], [20]. Building on this, we propose StrDLNN with an integrated friction modeling component.

Compared to existing approaches, it exhibits the following three key characteristics: (1) **Data requirements:** Traditional PINNs rely on  $f(q_n, \dot{q}_n, \ddot{q}_n) = \tau_n$ , which requires velocity and acceleration data. In contrast, our model uses position-only inputs to learn  $f(q_{n-1}, q_n, q_{n+1}) = \tau_n$  with high accuracy. (2) **Calculation process:** Traditional PINNs require computing the Hessian matrix  $(\frac{\partial^2 \mathcal{L}}{\partial \dot{q}^2})$  for each sample. StrDLNN avoids such second-order computations, reducing computational overhead and improving robustness. (3) **Friction model:** By integrating StrDLNN with a friction model, we obtain the dissipation-enhanced StrDLNN, which enables the accurate learning of both overall dynamics and friction parameters solely from position data, without requiring friction labels.

## III. PRELIMINARIES

### A. Continuous Euler-Lagrange Equations

Lagrangian mechanics begins with the Continuous d'Alembert Principle, from which the forced Euler-Lagrange equations are systematically derived using variational calculus, incorporating non-conservative forces into the formulation. The resulting equations take the following form:

$$\frac{d}{dt} \left( \frac{\partial \mathcal{L}(q, \dot{q})}{\partial \dot{q}} \right) - \frac{\partial \mathcal{L}(q, \dot{q})}{\partial q} = F(q, \dot{q}) \quad (1)$$

where  $\mathcal{L}(q, \dot{q})$  represents the Lagrangian function,  $F(q, \dot{q})$  represents the generalized non-conservative forces. In the absence

of such forces (i.e.,  $F = 0$ ), the equation reduces to the classical Euler–Lagrange equation. When  $F = \tau$  representing control inputs, the formulation becomes suitable for modeling controlled mechanical systems, and we further obtain:

$$\frac{\partial^2 \mathcal{L}(q, \dot{q})}{\partial^2 \dot{q}} \ddot{q} + \frac{\partial \mathcal{L}(q, \dot{q})}{\partial q \partial \dot{q}} \dot{q} - \frac{\partial \mathcal{L}(q, \dot{q})}{\partial q} = \tau \quad (2)$$

The Lagrangian is generally chosen to be

$$\mathcal{L}(q, \dot{q}) = T(q, \dot{q}) - V(q) = \frac{1}{2} \dot{q}^T M(q) \dot{q} - V(q) \quad (3)$$

with the kinetic energy  $T$ , the potential energy  $V$  and the mass matrix  $M(q)$ . Substituting  $\mathcal{L}$  with the kinetic and potential energy in (2), we obtain the following second-order ordinary differential equation:

$$M(q) \ddot{q} + \underbrace{\dot{M}(q) \dot{q} - \frac{1}{2} \left( \dot{q}^T \frac{\partial M(q)}{\partial q} \dot{q} \right)^T}_{=c(q, \dot{q})} + \underbrace{\frac{\partial V(q)}{\partial q}}_{=g(q)} = \tau \quad (4)$$

where  $\tau_I = M(q) \ddot{q}$  represents the inertial force,  $\tau_c = c(q, \dot{q})$  describes the forces generated by the Centripetal and Coriolis forces and  $\tau_g = g(q)$  represents the gravitational forces.

### B. Discrete Euler-Lagrange Equations

Based on the Discrete d'Alembert Principle, by uniformly partitioning the time interval  $[t_0, t_N]$  into  $N$  subintervals of equal length  $h$  (i.e., uniform time steps), the Lagrangian  $\mathcal{L}$  and force  $F$  are numerically approximated. The discrete forced Euler–Lagrange equations can then be derived by performing variational analysis on these discrete terms [9].

$$\nabla_2 \mathcal{L}_\Delta(q_{n-1}, q_n) + \nabla_1 \mathcal{L}_\Delta(q_n, q_{n+1}) + F_\Delta^+(q_{n-1}, q_n) + F_\Delta^-(q_n, q_{n+1}) = 0 \quad (5)$$

where  $n$  is the index of a specific subinterval, with  $n = 1, \dots, N-1$ , and  $N$  represents the total number of discretization steps for the entire interval.  $\mathcal{L}_\Delta$  denotes the discrete Lagrangian, and  $\nabla_1 \mathcal{L}_\Delta$  and  $\nabla_2 \mathcal{L}_\Delta$  denote the partial derivatives with respect to the first and second variables, respectively. In addition,  $F_\Delta^+(q_{n-1}, q_n)$  and  $F_\Delta^-(q_n, q_{n+1})$  represent the equivalent discrete actions acting on  $q_n$  over the intervals  $[t_{n-1}, t_n]$  and  $[t_n, t_{n+1}]$ , respectively. Using the midpoint rule for approximation, the discrete Lagrangian term and discrete force term can be expressed as:

$$\mathcal{L}_\Delta(q_n, q_{n+1}) := h \mathcal{L}\left(\frac{q_n + q_{n+1}}{2}, \frac{q_{n+1} - q_n}{h}\right),$$

$$F_\Delta^\pm(q_n, q_{n+1}) := \frac{h}{2} F\left(\frac{q_n + q_{n+1}}{2}, \frac{q_{n+1} - q_n}{h}\right) \quad (6)$$

where we have introduced the notation  $q_{n+} = \frac{1}{2}(q_n + q_{n+1})$ ,  $v_{n+} = \frac{1}{h}(q_{n+1} - q_n)$ ,  $q_{n-} = \frac{1}{2}(q_{n-1} + q_n)$ ,  $v_{n-} = \frac{1}{h}(q_n - q_{n-1})$  for brevity. By substituting (6) into (5) and performing further differentiation calculations [9]:

$$\begin{aligned} & \frac{h}{2} \nabla_1 \mathcal{L}(q_{n-}, v_{n-}) + \nabla_2 \mathcal{L}(q_{n-}, v_{n-}) + \frac{h}{2} \nabla_1 \mathcal{L}(q_{n+}, v_{n+}) \\ & - \nabla_2 \mathcal{L}(q_{n+}, v_{n+}) + \frac{h}{2} F(q_{n-}, v_{n-}) + \frac{h}{2} F(q_{n+}, v_{n+}) = 0 \end{aligned} \quad (7)$$

Unlike conventional methods that treat position and velocity as independent variables, discrete variational mechanics implicitly derives velocity from the position sequence. Each node  $n$  as the shared endpoint of two adjacent intervals, is associated with both  $v_{n-}$  and  $v_{n+}$ . Based on the variational principle, this approach preserves geometric structures like symplecticity, enhancing stability and accuracy in long-term predictions.

### C. Robot Dynamics Modeling

A robot dynamic model describes the relationship between joint torques and resulting motion. The general inverse dynamics model for a robot is given by

$$M(q) \ddot{q} + c(q, \dot{q}) + g(q) + \tau_f = \tau + \tau_f = \tau_u \quad (8)$$

The inverse dynamics can be represented as a one-dimensional vector diagram, as shown in Fig. 1(b), where the dynamic torque  $\tau = M(q) \ddot{q} + c(q, \dot{q}) + g(q)$  is expressed as the difference between the total torque  $\tau_u$  and the friction torque  $\tau_f$ , which serves as a simplified form of (4).

The dynamic torque  $\tau$  depends on a set of rigid body dynamics (RBD) parameters  $\theta_{\text{RBD}}$ . However, in practice, the torque predicted by (4) may deviate from the actual measurements due to: (i) misspecification of  $\theta_{\text{RBD}}$ , (ii) measurement errors in the observed values of  $q$ ,  $\dot{q}$ ,  $\ddot{q}$ , and  $\tau$ , and (iii) inaccurate modeling of physical phenomena such as joint friction [3].

For (i), PINNs use end-to-end data-driven modeling without requiring the known  $\theta_{\text{RBD}}$ . For (ii), the proposed StrDLNN avoids computing  $\dot{q}$  and  $\ddot{q}$  via differentiation, thereby reducing the impact of measurement errors to some extent. For (iii), our model introduces a friction model and the experimental results verify its capability to accurately capture joint friction  $\tau_f$ . For UR robots, the total torque  $\tau_u$  is not directly measured but is instead computed from motor currents  $I$ , for the  $j$ -th joint, the motor dynamics is given by:

$$\tau_{u_j} = r k_j I_j \quad (9)$$

where  $k_j$  is the torque constant of the motor at joint  $j$ , and  $r = 100$  is the gear ratio.

## IV. METHODOLOGY

### A. Discrete Dynamics Formula for Robots

Building on the discrete forced Euler–Lagrange equation in (7), we further derive the Discrete Dynamics Formula for Robots, which forms the foundation for constructing the proposed StrDLNN. As part of this process, a second-order Taylor expansion of the position is performed:

$$\begin{aligned} q_{n-1} &= q(t_n - h) = q_n - h \dot{q}_n + \frac{h^2}{2} \ddot{q}_n + o(h^3), \\ q_{n+1} &= q(t_n + h) = q_n + h \dot{q}_n + \frac{h^2}{2} \ddot{q}_n + o(h^3) \end{aligned} \quad (10)$$

Subsequently, a first-order Taylor expansion of the function  $F(q, v)$  at  $(q_n, \dot{q}_n)$  yields:

$$F(q, v) = F(q_n, \dot{q}_n) + \frac{\partial F}{\partial q}(q - q_n) + \frac{\partial F}{\partial v}(v - \dot{q}_n) + o(h^2) \quad (11)$$

Applying it to  $F(q_{n-}, v_{n-})$  and  $F(q_{n+}, v_{n+})$ , where the Taylor expansions of  $q_{n-}$ ,  $v_{n-}$ ,  $q_{n+}$ ,  $v_{n+}$  can be further derived from



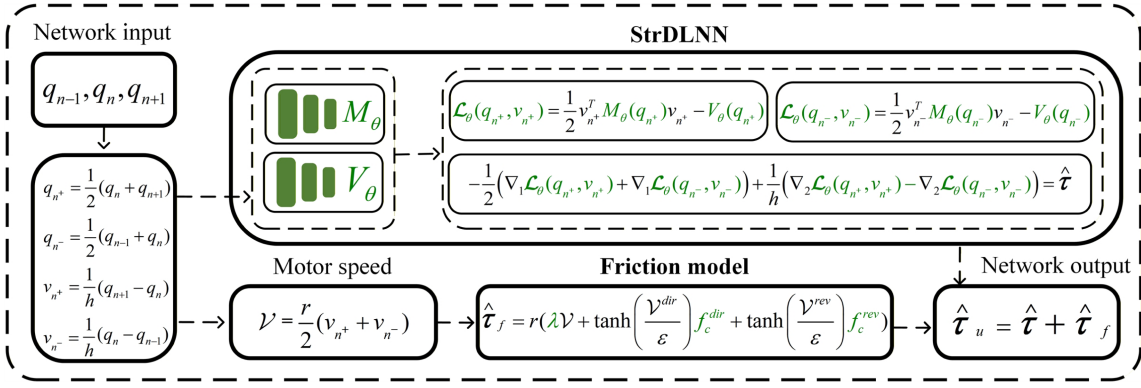


Fig. 2. Structure of the proposed StrDLNN and friction identification modules, along with their integration for robotic inverse dynamics modeling and prediction. Trainable components are highlighted in green.

(10), the two force terms combine to yield:

$$F(q_n^-, v_n^-) + F(q_n^+, v_n^+) = 2F(q_n, \dot{q}_n) + o(h^2) \quad (12)$$

Within second-order accuracy  $o(h^2)$ , the averaged discrete force term  $\frac{1}{2}(F(q_n^-, v_n^-) + F(q_n^+, v_n^+))$  can be approximated by  $F(q_n, \dot{q}_n)$ . This approximation is suitable for manipulator modeling, as forces vary smoothly over time, and with small  $h$ . Setting  $F = \tau$  as the control input yields:

$$\begin{aligned} & -\frac{1}{2}(\nabla_1 \mathcal{L}(q_n^-, v_n^-) + \nabla_1 \mathcal{L}(q_n^+, v_n^+)) \\ & + \frac{1}{h}(\nabla_2 \mathcal{L}(q_n^+, v_n^+) - \nabla_2 \mathcal{L}(q_n^-, v_n^-)) = \tau \end{aligned} \quad (13)$$

where  $\nabla_1 \mathcal{L}(q, v) = \frac{1}{2}v^T \frac{\partial M(q)}{\partial q} v - \frac{\partial V(q)}{\partial q}$ ,  $\nabla_2 \mathcal{L}(q, v) = M(q)v$ . Substituting these into (13) yields:

$$\begin{aligned} & \frac{1}{h}(M(q_n^+)v_{n^+} - M(q_n^-)v_{n^-}) \\ & - \frac{1}{4}\left(v_{n^-}^T \frac{\partial M(q_n^-)}{\partial q_{n^-}} v_{n^-} + v_{n^+}^T \frac{\partial M(q_n^+)}{\partial q_{n^+}} v_{n^+}\right) \\ & + \frac{1}{2}\left(\frac{\partial V(q_n^-)}{\partial q_{n^-}} + \frac{\partial V(q_n^+)}{\partial q_{n^+}}\right) = \tau \end{aligned} \quad (14)$$

Lagrangian-based methods embed physical structure, enabling force decomposition from inverse dynamics data [6], which is crucial for understanding the robot's dynamic properties [18]. To express these decomposed forces in discrete dynamics, we apply the midpoint value approximation to obtain  $\frac{1}{h}(M(q_n^+)v_{n^+} - M(q_n^-)v_{n^-}) \approx \frac{d}{dt}[M(q)v] \approx M(\frac{q_n^- + q_n^+}{2})\frac{v_{n^+} - v_{n^-}}{h} + \dot{M}(\frac{q_n^- + q_n^+}{2})\frac{v_{n^-} + v_{n^+}}{2}$ . Substituting this into (14) gives the detailed decomposition of each force

component:

$$\begin{aligned} \tau_I &= M\left(\frac{q_n^- + q_n^+}{2}\right) \frac{v_{n^+} - v_{n^-}}{h}, \\ \tau_c &= \dot{M}\left(\frac{q_n^- + q_n^+}{2}\right) \frac{v_{n^-} + v_{n^+}}{2} \\ & - \frac{1}{4}\left(v_{n^-}^T \frac{\partial M(q_n^-)}{\partial q_{n^-}} v_{n^-} + v_{n^+}^T \frac{\partial M(q_n^+)}{\partial q_{n^+}} v_{n^+}\right), \\ \tau_g &= \frac{1}{2}\left(\frac{\partial V(q_n^-)}{\partial q_{n^-}} + \frac{\partial V(q_n^+)}{\partial q_{n^+}}\right) \end{aligned} \quad (15)$$

The sum of the three torques represents the fundamental form of discrete forced Lagrangian dynamics for robotic systems and serves as the discrete counterpart to the continuous formulation given in (4).

### B. StrDLNN Network Structure

As shown in Fig. 2, to construct StrLNN, three generalized coordinates  $(q_{n-1}, q_n, q_{n+1})$  sampled at consecutive time intervals with  $h$ . These are first transformed via a feature mapping to produce  $(q_n^+, q_n^-, v_{n^+}, v_{n^-})$  which are then fed into the neural network for training. We use two neural networks with parameters  $\theta = (\theta_M, \theta_V)$  to approximate the mass matrix  $M_\theta$  and the potential energy  $V_\theta$ , respectively. To ensure that the learned mass matrix is symmetric positive definite,  $M_\theta$  is constructed via Cholesky decomposition:

$$M_\theta = L_\theta L_\theta^T + \varepsilon I = (l_{\theta_d} + l_{\theta_o})(l_{\theta_d} + l_{\theta_o})^T + \varepsilon I \quad (16)$$

where,  $L_\theta$  is a lower triangular matrix with positive diagonal elements. It is composed of two parts:  $l_{\theta_d}$ , which predicts the diagonal elements, and  $l_{\theta_o}$ , which predicts the off-diagonal elements. A small term  $\varepsilon I$ , where  $I$  is the identity matrix and  $\varepsilon$  is a small positive parameter, is added to ensure positive definiteness. Subsequently, the Lagrangian values at two states are computed based on  $M_\theta$  and  $V_\theta$ :

$$\begin{aligned} \mathcal{L}_\theta(q_{n^+}, v_{n^+}) &= \frac{1}{2}v_{n^+}^T M_\theta(q_{n^+})v_{n^+} - V_\theta(q_{n^+}) \\ \mathcal{L}_\theta(q_{n^-}, v_{n^-}) &= \frac{1}{2}v_{n^-}^T M_\theta(q_{n^-})v_{n^-} - V_\theta(q_{n^-}) \end{aligned} \quad (17)$$

Subsequently, the joint torques are predicted based on (13):

$$-\frac{1}{2}(\nabla_1 \mathcal{L}_\theta(q_{n+}, v_{n+}) + \nabla_1 \mathcal{L}_\theta(q_{n-}, v_{n-})) + \frac{1}{h}(\nabla_2 \mathcal{L}_\theta(q_{n+}, v_{n+}) - \nabla_2 \mathcal{L}_\theta(q_{n-}, v_{n-})) = \hat{\tau} \quad (18)$$

where  $\hat{\tau}$  denotes the predicted torque. Notably, unlike DeLaN and LNN, which require computing the second-order derivatives of the Lagrangian  $\mathcal{L}$  in (2) to predict joint torques, StrDLNN relies solely on first-order derivatives, thereby avoiding Hessian computation and resulting in a simpler computational process. To mitigate torque modeling errors in upper joints due to large inter-joint torque magnitude differences, a weighted mean squared error loss is used, where each weight is set as the inverse of the corresponding joint's variance to normalize errors across joints:

$$\text{loss}(\tau, \hat{\tau}) = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n \frac{(\tau_{ij} - \hat{\tau}_{ij})^2}{\sigma_j^2} \quad (19)$$

where  $\tau_{ij}$  and  $\hat{\tau}_{ij}$  are the ground-truth and predicted torques of the  $i$ -th sample at the  $j$ -th degree of freedom (DoF), respectively;  $\sigma_j^2$  is the torque variance at the  $j$ -th DoF;  $n$  is the number of DoFs; and  $m$  is the number of samples.

### C. Introducing Friction to Model Learning

In UR-series robots, friction mainly originates from the harmonic drive and depends on joint speed, rotation direction, and load. Given the lack of a universally accepted model, a trade-off between accuracy and complexity is necessary. Here, friction is modeled as the sum of a standard torque and an asymmetric smooth Coulomb term that depends on the direction of mechanical power flow rather than velocity sign. The joint friction torque for joint  $j$  is computed as [4]:

$$\tau_{f_j} = r \left( \lambda_j \vartheta_j + \tanh \left( \frac{\vartheta_j^{\text{dir}}}{\varepsilon} \right) f_{c_j}^{\text{dir}} + \tanh \left( \frac{\vartheta_j^{\text{rev}}}{\varepsilon} \right) f_{c_j}^{\text{rev}} \right) \quad (20)$$

with

$$\vartheta_j^{\text{dir}} = \begin{cases} \vartheta_j, & \text{if } \vartheta_j \cdot \tau_{u_j} > 0 \\ 0, & \text{if } \vartheta_j \cdot \tau_{u_j} < 0 \end{cases}, \quad \vartheta_j^{\text{rev}} = \begin{cases} \vartheta_j, & \text{if } \vartheta_j \cdot \tau_{u_j} < 0 \\ 0, & \text{if } \vartheta_j \cdot \tau_{u_j} > 0 \end{cases}$$

Here, the constant  $\varepsilon$  is a smoothing parameter empirically set to  $10^{-2}$ , and  $\lambda_j$  denotes the viscous friction coefficient. The Coulomb friction term takes the value  $f_{c_j}^{\text{dir}}$  when power flows from the motor to the load, and  $f_{c_j}^{\text{rev}}$  when the power flow is reversed. The motor speed is given by  $\vartheta_j = r \cdot \dot{q}_j$ , where  $\dot{q}_j$  denotes the joint velocity, and  $r = 100$  is the gear ratio. Unlike traditional friction models that compute friction torque based on joint velocity  $\dot{q}$ , we approximate  $\dot{q}$  from position  $q$  and then compute the motor speed  $\vartheta$ . The motor speed is given by:

$$\vartheta = \frac{r}{2}(v_{n+} + v_{n-}) \quad (21)$$

The friction coefficients are denoted as  $\theta_f = (\lambda, f_c^{\text{dir}}, f_c^{\text{rev}})$ , which are treated as learnable parameters and optimized as network weights. The motor speed  $\vartheta$  is subsequently used as the input to the friction model to predict and update the friction torque  $\hat{\tau}_f$ . The friction model is then integrated with StrDLNN to predict the total torque as  $\hat{\tau}_u = \hat{\tau} + \hat{\tau}_f$ . The loss function

defined in (19) is used to compare the predicted total torque  $\hat{\tau}_u$  with the ground-truth torque  $\tau_u$ , and serves to optimize the entire model.

## V. EXPERIMENTS AND DISCUSSION

### A. Experiment Setup

1) *Evaluation Metrics*: In the experiments, we use normalized mean square error (nMSE), and the coefficient of determination ( $R^2$ ) to evaluate model performance.  $R^2$  is defined as  $R^2 = 1 - \frac{\sum_{j=1}^n \sum_{i=1}^N (\tau_{ij} - \bar{\tau}_{ij})^2}{\sum_{j=1}^n \sum_{i=1}^N (\tau_{ij} - \bar{\tau}_j)^2}$ , where  $\bar{\tau}$  denotes the mean torque,  $n$  represents the number of joints, and  $N$  indicates the number of samples per joint.  $R^2$  ranges from 0 to 1, with higher values indicating better predictions.

2) *Compared Baselines*: To demonstrate the effectiveness of the proposed StrDLNN model in modeling the robot dynamics  $f(q_{n-1}, q_n, q_{n+1}) = \tau_n$ , we use several baseline approaches. Details are provided below:

- *MLP*: A black-box model without any prior knowledge, using only position data as input to predict the robot's dynamics  $f(q_{n-1}, q_n, q_{n+1}) = \tau_n$ .
- *DiffNEA* [5]: A parameter identification method that learns the links' mass, inertia matrix, and transform as trainable parameters, enforces physical constraints, and computes  $f(q_n, \dot{q}_n, \ddot{q}_n) = \tau_n$  via RNEA.
- *LNN* [8]: Uses a single neural network to predict the robot's unstructured  $\mathcal{L}$ , and then computes the dynamics  $f(q_n, \dot{q}_n, \ddot{q}_n) = \tau_n$  according to (2).
- *DeLaN* [7]: Uses two networks to model the kinetic energy  $T$  and potential energy  $V$ , forming the structured  $\mathcal{L} = T - V$ , and then computes the dynamics  $f(q_n, \dot{q}_n, \ddot{q}_n) = \tau_n$  according to (2).
- *DFLNN* [9]: Uses a single neural network to predict the robot's unstructured  $\mathcal{L}$ , and then computes the dynamics  $f(q_{n-1}, q_n, q_{n+1}) = \tau_n$  according to (13).

3) *Implementation Details*: In each experiment, the dynamics models are trained on a fixed training dataset until convergence on the validation set, then evaluated on a separate test set. To reduce variability from random initialization, results are averaged over 5 runs with different seeds. All neural networks are implemented in JAX, using automatic differentiation for partial derivatives. The ADAM optimizer is applied with a batch size of 512, learning rate of  $10^{-4}$ , and weight decay of  $10^{-5}$ . The activation function is *softplus*, and each network has two layers with 64 neurons per layer.

### B. Simulation Experiments

In simulation, we evaluate StrDLNN's performance with ideal data, crucial for its physical consistency and credibility, where better performance indicates greater adherence to physical laws [6], [19]. We assess its ability to recover the robot's dynamics from position data alone, comparing data efficiency, sensitivity to noise, extrapolation, and robustness to sampling frequency with baseline models.

1) *Dataset Construction*: We simulate a two-link pendulum with continuous revolute joints in the vertical  $x$ - $z$  plane under gravity, using the PyBullet physics engine and its inverse dynamics model based on (4). Without dissipative effects, the system serves as an ideal benchmark for evaluating the dynamics in (4).

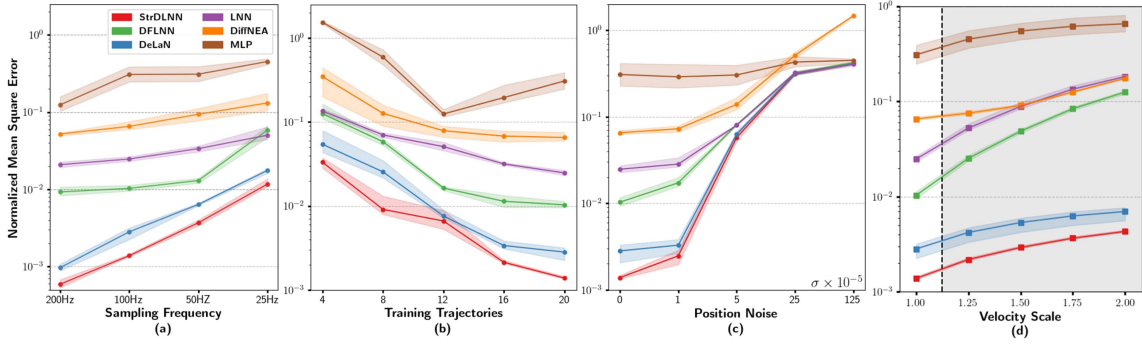


Fig. 3. Model performance under different conditions: solid lines show the mean over 5 seeds. (a) sampling frequency, (b) training trajectories, (c) position noise, (d) velocity extrapolation.

TABLE I

FORCE DECOMPOSITION RESULTS USING NMSE. IN THE LEGEND, GREEN, ORANGE, AND BLUE INDICATE THE BEST, SECOND-BEST, AND THIRD-BEST PERFORMANCES, RESPECTIVELY.

| Models        | Inertial Torque $\tau_I$                | Coriolis Torque $\tau_c$                | Gravitational Torque $\tau_g$           |
|---------------|-----------------------------------------|-----------------------------------------|-----------------------------------------|
| MLP           | 1.35e+0 $\pm$ 1.03e+0                   | 3.08e+0 $\pm$ 1.24e+0                   | 9.28e-2 $\pm$ 7.43e-2                   |
| DiffNEA       | 1.34e-1 $\pm$ 8.61e-2                   | 5.97e-1 $\pm$ 1.66e-2                   | 1.22e-2 $\pm$ 8.52e-3                   |
| LNN           | <b>2.63e-2 <math>\pm</math> 7.79e-3</b> | <b>1.46e-1 <math>\pm</math> 2.89e-2</b> | <b>9.14e-4 <math>\pm</math> 2.57e-4</b> |
| DFLNN         | 1.04e-1 $\pm$ 3.53e-2                   | 2.57e-1 $\pm$ 2.56e-1                   | 5.30e-2 $\pm$ 4.52e-2                   |
| DeLaN         | <b>1.01e-2 <math>\pm</math> 2.41e-3</b> | <b>1.90e-2 <math>\pm</math> 8.15e-3</b> | <b>3.96e-4 <math>\pm</math> 8.63e-5</b> |
| StrDLNN(ours) | <b>2.27e-3 <math>\pm</math> 6.40e-5</b> | <b>9.06e-3 <math>\pm</math> 9.72e-4</b> | <b>1.07e-4 <math>\pm</math> 1.92e-5</b> |

Trajectory planning follows sinusoidal functions with 26 random initial and final joint angle sets for the two joints. At a sampling frequency of 100 Hz, each 2 s trajectory records joint positions  $q$  and torques  $\tau$ , while velocities  $\dot{q}$  and accelerations  $\ddot{q}$  are computed using finite differences and low-pass filters. From these trajectories, two corresponding datasets,  $(q_n, \dot{q}_n, \ddot{q}_n, \tau_n)$  and  $(q_{n-1}, q_n, q_{n+1}, \tau_n)$ , are constructed and employed according to the model requirements in Section V-A2. Out of the 26 trajectories, 20 are randomly selected for training, 3 for validation, and 3 for testing.

2) *Torque Decomposition*: All data-driven models use the loss in (19) to fit predicted  $\hat{\tau}$  to the ground truth without explicitly decomposing the forces. However, Lagrangian-based methods can infer inertial torque  $\tau_I$ , Coriolis torque  $\tau_c$ , and gravitational torque  $\tau_g$  by embedding physical structure, with decomposition accuracy reflecting the model's ability to capture the robot's underlying dynamics. DeLaN and LNN compute the decomposed torques using the continuous Euler–Lagrange formulation (4), while DFLNN and StrDLNN use the discrete formulation (15).

As shown in Table I, MLP, lacking physical priors, performs the worst. StrDLNN outperforms all models, followed by DeLaN, both using structured Lagrangians  $\mathcal{L}$  for better dynamics learning than the unstructured  $\mathcal{L}$  in DFLNN and LNN. Despite using an unstructured  $\mathcal{L}$ , DFLNN performs worse than LNN. Overall, StrDLNN's structured  $\mathcal{L}$  enables it to recover robot dynamics from position data alone, even with the discrete Euler–Lagrange equation.

3) *Sampling Frequency and Data Efficiency*: The time interval between samples, determined by the step size  $h$ , affects both differentiation accuracy and numerical integration precision (5) and (12). A larger  $h$  increases difference errors, introducing data noise in methods that rely on this data, and reduces the

accuracy of discrete Forced Euler–Lagrange solutions. Since  $h$  is determined by the sampling frequency, we test with frequencies of 200, 100, 50, and 25 Hz. Results are shown in Fig. 3(a). All models show degraded performance as the sampling frequency decreases. For continuous Lagrangian-based methods, modeling accuracy is consistently lower at lower sampling frequencies compared to their discrete counterpart, highlighting the impact of step size  $h$ . StrDLNN consistently performs best, followed by DeLaN, and is less sensitive to  $h$  compared to methods that rely on differentiated data.

Fig. 3(b) shows that more training trajectories improve all models, with StrDLNN and DeLaN benefiting the most, followed by DFLNN, LNN, and DiffNEA. This suggests that structured  $\mathcal{L}$  utilize data more efficiently. In contrast, MLP starts to overfit after 12 trajectories. StrDLNN also achieves a larger nMSE reduction with the same additional data, indicating higher data efficiency.

4) *Position Noise and Velocity Extrapolation*: In real-world applications, position observations are always noisy, motivating sensitivity experiments on position noise. Zero-mean Gaussian noise  $\mathcal{N}(0, \sigma^2)$  is added to the position data to simulate position measurement noise. As shown in Fig. 3(c), as the standard deviation  $\sigma$  increases, the performance of all models deteriorates. StrDLNN achieves the lowest nMSE when  $\sigma < 5 \times 10^{-5}$ , with structured Lagrangian methods outperforming unstructured counterparts. At higher noise levels, Lagrangian-based methods stabilize and achieve performance comparable to that of MLP, while DiffNEA performs the worst. This indicates that, unlike PINNs, system identification methods are more sensitive to data quality.

In Fig. 3(d), we test extrapolation by training on  $1 \times$  velocity and evaluating at higher velocities. LNN and DFLNN degrade more sharply than DeLaN and StrDLNN. The robustness of DeLaN and StrDLNN arises from their structured  $\mathcal{L}$ , which predicts  $M_\theta(q)$  and  $V_\theta(q)$  solely from  $q$ , thereby enabling stronger velocity extrapolation. In contrast, LNN predicts an unstructured  $\mathcal{L}_\theta(q, \dot{q})$  from both  $q$  and  $\dot{q}$ , making it more sensitive to velocity variations.

### C. Physical Experiments

For the physical UR10e robot, we investigate the optimal integration of dissipative mechanisms into StrDLNN and assess whether coupling it with a friction model enables simultaneous accurate dynamics modeling and friction coefficient identification.



TABLE II

MAIN RESULTS ON THE UR10E IN MODELING THE TOTAL JOINT TORQUE  $\tau_u$  FOR EACH JOINT USING NMSE, AND REPORTING THE  $R^2$  SCORE FOR EACH MODEL, WITH CONFIDENCE INTERVALS AVERAGED OVER 5 SEEDS. IN THE LEGEND, GREEN, ORANGE, AND BLUE INDICATE THE BEST, SECOND-BEST, AND THIRD-BEST PERFORMANCES, RESPECTIVELY.

| Models                 | Joint 0                  | Joint 1                  | Joint 2                  | Joint 3                  | Joint 4                  | Joint 5                  | $R^2$                    |
|------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| MLP                    | <b>1.84e-1 ± 2.46e-2</b> | <b>9.58e-2 ± 5.51e-3</b> | 8.54e-2 ± 1.30e-2        | <b>6.53e-2 ± 4.06e-3</b> | <b>5.47e-2 ± 4.64e-3</b> | <b>4.99e-2 ± 3.20e-3</b> | <b>9.11e-1 ± 6.35e-3</b> |
| DiffNEA                | 8.07e-1 ± 6.41e-2        | 2.16e-1 ± 7.52e-2        | 1.73e-1 ± 2.78e-2        | 9.20e-1 ± 9.05e-2        | 1.01e+0 ± 5.53e-1        | 1.04e+0 ± 5.15e-1        | 2.60e-1 ± 2.32e-2        |
| LNN                    | 9.17e-1 ± 9.59e-3        | 2.29e-1 ± 1.42e-2        | 1.89e-1 ± 1.89e-2        | 9.15e-1 ± 8.04e-3        | 8.73e-1 ± 1.73e-2        | 9.95e-1 ± 3.44e-2        | 3.14e-1 ± 5.61e-3        |
| DFLNN                  | 8.32e-1 ± 3.49e-1        | 2.42e-1 ± 9.85e-4        | 2.05e-1 ± 3.90e-3        | 9.27e-1 ± 1.70e-2        | 9.52e-1 ± 1.28e-2        | 1.03e+0 ± 9.82e-3        | 2.73e-1 ± 3.99e-3        |
| DeLaN                  | 8.95e-1 ± 1.29e-2        | 2.35e-1 ± 1.95e-2        | 2.13e-1 ± 2.00e-2        | 9.46e-1 ± 1.33e-2        | 2.53e+0 ± 3.19e+0        | 1.10e+0 ± 1.88e-2        | 2.85e-1 ± 5.97e-3        |
| StrDLNN(ours)          | 1.00e+0 ± 4.43e-3        | 2.50e-1 ± 3.13e-3        | 1.97e-1 ± 3.43e-3        | 1.00e+0 ± 1.67e-2        | 9.27e-1 ± 7.97e-3        | 1.04e+0 ± 4.50e-3        | 2.64e-1 ± 2.96e-3        |
| StrDLNN-Damping        | 2.55e-1 ± 8.73e-3        | 2.56e-1 ± 5.53e-3        | <b>5.21e-2 ± 7.11e-3</b> | 1.71e-1 ± 8.57e-3        | 2.62e-1 ± 8.09e-3        | 2.71e-1 ± 6.22e-3        | 7.89e-1 ± 4.11e-3        |
| StrDLNN-MLP            | <b>1.68e-1 ± 1.41e-2</b> | <b>8.89e-2 ± 2.38e-3</b> | <b>5.01e-2 ± 1.11e-2</b> | <b>5.79e-2 ± 8.27e-3</b> | <b>6.09e-2 ± 4.45e-3</b> | <b>5.94e-2 ± 8.04e-3</b> | <b>9.19e-1 ± 5.23e-3</b> |
| StrDLNN-Friction(ours) | <b>1.25e-1 ± 1.19e-3</b> | <b>5.97e-2 ± 3.98e-4</b> | <b>2.39e-2 ± 8.87e-4</b> | <b>6.26e-2 ± 2.39e-3</b> | <b>3.73e-2 ± 1.91e-3</b> | <b>3.19e-2 ± 9.28e-4</b> | <b>9.43e-1 ± 9.45e-4</b> |

TABLE III

IDENTIFIED FRICTION PARAMETERS (LEFT: LEAST SQUARES [4]; RIGHT: STRDLNN-FRICTION(OURS))

| Joint | $\lambda$ [Nm s/rad] |        | $f_c^{\text{dir}}$ [Nm] |        | $f_c^{\text{rev}}$ [Nm] |        |
|-------|----------------------|--------|-------------------------|--------|-------------------------|--------|
| 0     | 3.3e-3               | 2.9e-3 | 2.0e-1                  | 2.3e-1 | 2.9e-2                  | 2.3e-5 |
| 1     | 2.4e-3               | 2.6e-3 | 2.9e-1                  | 2.5e-1 | 1.7e-2                  | 6.3e-2 |
| 2     | 1.0e-3               | 1.1e-3 | 1.5e-1                  | 1.3e-1 | 6.8e-3                  | 3.7e-2 |
| 3     | 3.5e-4               | 3.4e-4 | 3.4e-2                  | 3.4e-2 | 2.0e-2                  | 1.5e-2 |
| 4     | 8.0e-4               | 7.9e-4 | 7.1e-2                  | 7.1e-2 | 4.3e-5                  | 6.7e-6 |
| 5     | 7.6e-4               | 7.3e-4 | 7.7e-2                  | 7.8e-2 | 1.1e-5                  | 1.9e-5 |

1) *Parameter Identification*: The UR10e, Universal Robots' manipulator with 6-DoF, is ideal for testing the dynamics in (8) due to its pronounced friction effects. Since the manufacturer does not provide the motor torque constant  $k$  in (9) and the friction coefficients ( $\lambda$ ,  $f_c^{\text{dir}}$ ,  $f_c^{\text{rev}}$ ) in (20). Before modeling, we need to identify these parameters from data. Following Ref. [4], which identifies friction parameters and torque constants from the same dataset, we used its data acquisition and processing procedures to estimate the UR10e's torque constant and friction coefficients via least squares. The friction coefficients (Table III) summarize our identification results.

2) *Dataset Construction*: The excitation trajectories, generated from a fifth-order Fourier series, are designed to maximize model activation during robot motion. This experiment collects the robot's kinematic states and joint currents for subsequent modeling and learning [1]. The motion of the  $j$ -th joint is expressed as

$$q_j^t = q_j^0 + \sum_{k=1}^5 \left( \frac{a_j^k}{w_f^k} \sin(w_f^k t) - \frac{b_j^k}{w_f^k} \cos(w_f^k t) \right) \quad (22)$$

where  $t$  denotes time,  $q_j^0$  represents the initial position,  $a_j^k$  and  $b_j^k$  denote the  $k$ -th order trajectory coefficients.  $w_f^k$  signifies the fundamental frequency. Typically, the fundamental frequency is set to 0.1 Hz. The trajectory coefficients are determined through an iterative optimization technique [21].

We generate 46 excitation trajectories, each 10 s long and sampled at 500 Hz, recording joint positions, velocities, and currents. Notably, the velocities are derived from the position data using the manufacturer's differentiation algorithm. Subsequently, accelerations are computed using finite differences and low-pass filters, and the total joint torques  $\tau_u$  are computed from (9).

Three trajectories are reserved for testing, three for validation, and the remaining ones are used for training. Each trajectory is downsampled by half to reduce training time. From these trajectories, two datasets,  $(q_n, \dot{q}_n, \ddot{q}_n, \tau_n)$  and  $(q_{n-1}, q_n, q_{n+1}, \tau_n)$ , are constructed and used according to the model requirements in Section V-A2.

3) *Dissipation-Enhanced StrDLNN*: The Euler-Lagrange equation and classical Lagrangian dynamics assume a conservative system; although non-conservative forces can be added via a forced formulation, complex friction is still hard to capture. As shown in Table II, the Lagrangian-based methods and DiffNEA underperform MLP due to their inadequate capability to learn non-conservative forces. This motivates the integration of an optimal energy dissipation mechanism into StrDLNN to simultaneously model both the overall dynamics and the frictional effects in equation (8).

Three models are developed: (1) **StrDLNN-MLP**, using a black-box to capture input-dependent non-conservative and other dissipative forces [2]; (2) **StrDLNN-Damping**, training a semi-definite damping matrix to compute dissipation forces [15]; and (3) **StrDLNN-Friction**, integrating StrDLNN with a Non-Symmetrical Coulomb Friction model (Fig. 2). As shown in Table II, StrDLNN-Damping remains less accurate than MLP, while StrDLNN-MLP outperforms MLP overall but underperforms on some joints. Both variants increase model complexity by adding an extra network, with StrDLNN-Damping further computing a semi-definite matrix and StrDLNN-MLP introducing a black-box dissipation mechanism that reduces PINN transparency. StrDLNN-Friction, by contrast, attains the best accuracy ( $R^2$  up to 0.94) with only a few additional parameters, making it the most effective dissipation-enhanced StrDLNN.

4) *Torque Decomposition and Friction Identification*: Accurately estimating each component of the robot's dynamics in (8) is crucial for understanding its characteristics. Since the manufacturer provides no ground truth for force decomposition, we use system identification to obtain the decomposed forces. Ref. [4] provides a method for separating the total torque  $\tau_u$  into friction torque  $\tau_f$  and dynamic torque  $\tau$  for the UR robot, we then proceed to identify  $\tau_I$ ,  $\tau_c$  and  $\tau_g$  within  $\tau$  using DiffNEA. The experimental results in Table IV compare StrDLNN-Friction, DeLaN-Friction (which uses the same friction model as defined in (20)), and MLP, with system identification predictions serving as the ground truth. It is noted that MLP is unable to predict friction torque.

TABLE IV  
FORCE DECOMPOSITION RESULTS USING NMSE IN THE UR10e EXPERIMENT,  
WITH BOLD REPRESENTING RESULTS CLOSER TO SYSTEM IDENTIFICATION

| Model    | StrDLNN-Friction                                      | DeLaN-Friction                                        | MLP                                 |
|----------|-------------------------------------------------------|-------------------------------------------------------|-------------------------------------|
| $\tau_I$ | $8.92\text{e-}1 \pm 1.44\text{e-}3$                   | <b><math>1.71\text{e-}1 \pm 3.98\text{e-}2</math></b> | $1.05\text{e}+0 \pm 1.41\text{e-}2$ |
| $\tau_C$ | $7.86\text{e-}1 \pm 6.95\text{e-}4$                   | <b><math>4.23\text{e-}1 \pm 4.90\text{e-}2</math></b> | $7.41\text{e}+2 \pm 2.76\text{e}+1$ |
| $\tau_g$ | <b><math>1.96\text{e-}1 \pm 1.23\text{e-}2</math></b> | $2.46\text{e-}1 \pm 5.88\text{e-}2$                   | $4.49\text{e-}1 \pm 2.52\text{e-}2$ |
| $\tau_f$ | <b><math>1.18\text{e-}1 \pm 2.95\text{e-}4</math></b> | $1.60\text{e-}1 \pm 1.55\text{e-}3$                   | —                                   |
| $\tau_u$ | $4.54\text{e-}2 \pm 3.01\text{e-}4$                   | <b><math>3.16\text{e-}2 \pm 6.67\text{e-}4</math></b> | $8.34\text{e-}2 \pm 3.31\text{e-}3$ |

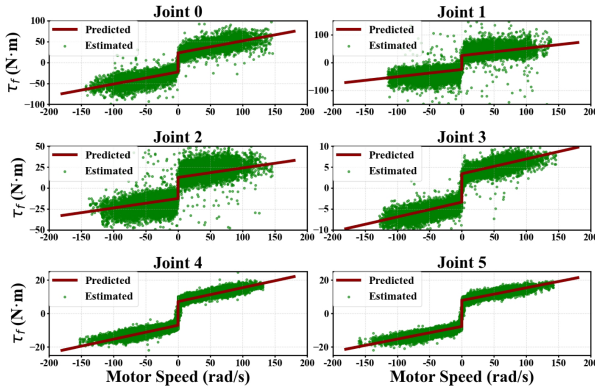


Fig. 4. Friction torque prediction results for robot joints. To ensure a one-to-one mapping between motor speed and predicted friction (20), only samples where motor speed and  $\tau_u$  share the same sign are shown (speed  $> 0 \Rightarrow \tau_u > 0$ , speed  $< 0 \Rightarrow \tau_u < 0$ ).

Overall, DeLaN-Friction performs best, slightly ahead of StrDLNN-Friction, and is better at predicting  $\tau_I$  and  $\tau_C$ , with  $\tau_I$  benefiting from the advantage of continuous Lagrangian methods due to its direct relation to acceleration. However, StrDLNN-Friction outperforms DeLaN-Friction in predicting  $\tau_f$  and  $\tau_g$  which is crucial for heavy-duty robotic arms [18].

The friction model of StrDLNN-Friction follows (20), with three parameters to be trained for each of the six joints:  $\lambda$ ,  $f_c^{\text{dir}}$ , and  $f_c^{\text{rev}}$ . These are modeled as part of the overall dynamics in (8) to predict the total torque  $\tau_u$ , without requiring labeled friction data for training. After training, the identified parameters are compared with friction coefficients obtained via the conventional least squares method using labeled friction data [4], as shown in Table III. The identification results of the two methods are very close. Fig. 4 illustrates the strong fitting performance of StrDLNN-Friction for the friction component, whose distribution with respect to motor speed matches typical friction characteristics.

## VI. CONCLUSION

This paper presents StrDLNN, based on the discrete Lagrangian formulation. In the two-link pendulum experiment, StrDLNN successfully recovered the underlying dynamics, excelling in data efficiency, extrapolation, and robustness to sampling frequency. Applied to the UR10e robot, it shows that integrating StrDLNN with a friction model enables accurate identification of robot dynamics and friction coefficients using

only position data, without friction labels. Future research could explore applying StrDLNN to more robotic platforms, such as ground vehicles [16] and drones [17], and leverage this model to enhance robotic control performance.

## REFERENCES

- [1] H. Hu, Z. Shen, and C. Zhuang, "A PINN-based friction-inclusive dynamics modeling method for industrial robots," *IEEE Trans. Ind. Electron.*, vol. 72, no. 5, pp. 5136–5144, May 2025.
- [2] Z. Li, S. Wu, W. Chen, and F. Sun, "Physics-informed neural networks for compliant robotic manipulators dynamic modeling," *J. Comput. Sci.*, vol. 90, 2025, Art. no. 102633.
- [3] M. Trinh, A. R. Geist, J. Monnet, S. Vilceanu, S. Trimpe, and C. Brecher, "Newtonian and Lagrangian neural networks: A comparison towards efficient inverse dynamics identification," *IFAC-PapersOnLine*, vol. 59, pp. 31–36, 2025.
- [4] E. Clochiatti, L. Scalera, P. Boscariol, and A. Gasparetto, "Electro-mechanical modeling and identification of the UR5 E-Series robot," *Robotica*, vol. 42, pp. 2430–2452, 2024.
- [5] G. Sutanto et al., "Encoding physical constraints in differentiable newton-euler algorithm," in *Proc. Conf. Learn. Dyn. Control*, 2020, pp. 804–813.
- [6] M. Lutter and J. Peters, "Combining physics and deep learning to learn continuous-time dynamics models," *Int. J. Robot. Res.*, vol. 42, pp. 83–107, 2023.
- [7] M. Lutter, C. Ritter, and J. Peters, "Deep Lagrangian networks: Using physics as model prior for deep learning," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–17.
- [8] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, "Lagrangian neural networks," in *Proc. ICLR Workshop Integration Deep Neural Models Differ. Equ.*, 2020, pp. 1–9.
- [9] M. D. Hansen, E. Celledoni, and B. K. Tapley, "Learning mechanical systems from real-world data using discrete forced Lagrangian dynamics," 2025, *arXiv:2505.20370*.
- [10] L. Liu, G. Zuo, J. Li, and J. Li, "Dynamics modeling with realistic constraints for trajectory tracking control of manipulator," in *Proc. IEEE Int. Conf. Real-Time Comput. Robot.*, 2022, pp. 99–104.
- [11] C. Feng et al., "Physics-informed neural network with physically consistent and residual learning for excavator precision operation control," *Appl. Soft Comput.*, vol. 167, 2024, Art. no. 112402.
- [12] R. Xu et al., "Velocity-inferred Hamiltonian neural networks: Learning energy-conserving dynamics from position-only data," 2025, *arXiv:2505.02321*.
- [13] C. Offen, "Machine learning of continuous and discrete variational odes with convergence guarantee and uncertainty quantification," *Math. Computation*, 2025.
- [14] M. Lahoud, G. Marchello, M. D'Imperio, A. Müller, and F. Cannella, "A deep learning framework for non-symmetrical coulomb friction identification of robotic manipulators," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 10510–10516.
- [15] J. Liu, P. Borja, and C. Della Santina, "Physics-informed neural networks to model and control robots: A theoretical and experimental investigation," *Adv. Intell. Syst.*, vol. 6, 2024, Art. no. 2300385.
- [16] W. Yang et al., "Trajectory tracking control of autonomous vehicles based on Lagrangian neural network dynamics model," *Proc. Inst. Mech. Engineers, Part D: J. Automobile Eng.*, vol. 238, pp. 3483–3498, 2024.
- [17] B. Breese, M. Kumar, M. Bolender, and D. W. Casbeer, "Physics-based neural networks for modeling & control of aerial vehicles," in *Proc. Amer. Control Conf.*, 2022, pp. 3218–3223.
- [18] X. Liang, T. Gao, Z. Yao, and J. Yao, "Dynamics modeling of robot manipulators based on deep Lagrangian network and torque separation technique," *Int. J. Mech. Syst. Dyn.*, 2025.
- [19] T. Duong, A. Altawaitan, J. Stanley, and N. Atanasov, "Port-Hamiltonian neural ODE networks on Lie groups for robot dynamics learning and control," *IEEE Trans. Robot.*, vol. 40, pp. 3695–3715, 2024.
- [20] Y. Lishkova et al., "Discrete Lagrangian neural networks with automatic symmetry discovery," *IFAC-PapersOnLine*, vol. 56, pp. 3203–3210, 2023.
- [21] V. Bonnet, P. Fraisse, A. Crosnier, M. Gautier, A. González, and G. Venture, "Optimal exciting dance for identifying inertial parameters of an anthropomorphic structure," *IEEE Trans. Robot.*, vol. 32, no. 4, pp. 823–836, Aug. 2016.